Consignes générales pour les TDs

FAITES BIEN LES SCHEMAS DEMANDES, **CE N'EST PAS OPTIONNEL** ET VOUS AIDERA À VISUALISER ET COMPRENDRE LES ALGORITHMES

POUR LES EXERCICES COMPORTANT DE LA REDACTION DE CODE EN C, **UTILISEZ UN EDITEUR DE TEXTE OU UN IDE**, AFIN DE POUVOIR RECUPERER VOS TRAVAUX POUR LES TPs.

Les exercices précédés d'un symbole losange ◆ sont obligatoires.

TD3 : Piles et Files

Thème 1 – Fonctions de la pile

Exercice 1) complément de cours – Piles

Partie I) Piles utilisant une liste chaînée pour stocker les valeurs

Rappel : dans ce cas, le type t_stacklist (pile) est le même que le type t_list (liste simple)

Question 1) Ecrivez la fonction pop() qui retire le premier élément de la pile. On considère que la pile contient au moins un élement.

Question 2) Ecrivez la fonction top() qui permet de consulter le premier élément de la pile

Partie II) Piles utilisant des tableaux pour stocker les valeurs

Rappel : dans ce cas, le type `t_stacktab` (pile) est défini par un tableau statique et sa taille logique.

Question 1) Écrivez la fonction `pop()` qui retire le dernier élément ajouté à la pile. On considère que la pile contient au moins un élément.

Question 2) Écrivez la fonction `top()` qui permet de consulter le dernier élément ajouté

Exercice 2) Escape game avec une pile

Pour cet exercice, vous pouvez utiliser les fonctions : push(), pop(), top(), isEmptyStack().

L'implémentation détaillée de la pile (liste ou tableaux) n'a aucune importance.

Les éléments à stocker dans la pile sont des **énigmes** : une **énigme** possède les champs :

1. **`int num;`** Un numéro d'ordre (de 1 jusque N s'il y N énigmes dans le jeu)
2. **`char *pass;`** Une clé d'entrée : un mot de passe pour pourvoir y accéder
3. **`char *question;`** Une question
4. **`char *answer;`** Une réponse attendue
5. **`char *reward;`** Une clé de sortie : un mot de passe pour accéder à l'énigme suivante

On utilisera le type **`: t_enigme`**, qui est une structure regroupant ces champs.

On utilisera la variable **`mygame`**, qui est une pile (**`stack`**) stockant des éléments de type **`t_enigme`**.

Question 1) en utilisant **`push`**(), **`pop`**() et **`top`**(), vérifiez , à l'aide d'une fonction '**`checkEscape'`** que l'escape game est bien construit, c'est-à-dire que le mot de passe '**`reward'`** d'une énigme donné bien accès à l'énigme suivante (**`pass`**).

Question 2) Ecrivez la fonction qui permet de dérouler le jeu entier.

Exercice 3): Vérifier qu'un mot est un palindrome en utilisant une pile de caractères.

Soit une chaîne de caractères **`myword`** dont on connait la taille logique **`size`**.

Ecrivez une fonction qui vérifie si cette chaîne est un palindrome en utilisant une pile de caractère. Pour créer cette pile, vous pouvez utiliser la fonction **`createStack`**() qui crée une pile vide.

Thème 2 – Fonctions sur les files

Exercice 4) Files – complément de cours

Partie I) Files utilisant une liste 'ht'

Question 1) Rappelez pourquoi on préfère utiliser une liste 'ht' plutôt qu'un liste simple pour implémenter une file

Question 2) Ecrivez la fonction **`enqueue`**(), qui ajoute un valeur en fin de file, c'est-à-dire à la fin de la liste ht
Question 3) Ecrivez la fonction **`dequeue`**(), qui retire la valeur en début de file.

Partie II) Files utilisant un tableau avec deux indices, first et last

Question 1) Faites le schéma d'une file d'entiers, avec un tel tableau, après avoir réalisé les opérations suivantes :

Enfiler 4, enfiler 8 , enfiler -42, défiler, enfiler 9, défiler, enfiler 0

Question 2) rappelez les conditions à tester pour savoir que la file est vide, puis que la file est pleine, dans le cas d'un buffer circulaire.

◆ *Exercice 5) utilisation d'une pile pour vérifier la validité d'une expression*

Les piles sont très souvent utilisées dans les outils d'analyse lexicale et syntaxique, utilisés par les interpréteurs et compilateurs.

On souhaite vérifier qu'une expression (mathématique) est correctement écrite, étape par étape

Pour cet exercice, une expression sera une chaîne de caractères (donc un tableau de caractères, dont on peut connaître la taille en utilisant la fonction **`strlen`**()).

Etape 1 : les parenthèses

On cherche à vérifier qu'il y a autant de parenthèses ouvrantes que de parenthèses fermantes dans l'expression, et qu'une parenthèse fermante correspond à une parenthèse ouvrante précédente

Exemples :

`(3+x)`	est une expression bien parenthésée
`((2*5)+1`	ne l'est pas
`(6*9)-)7+2(`	ne l'est pas, même s'il y a autant de parenthèses ouvrantes que de fermantes

<u>Question 1)</u> Proposez une fonction 'simple' pour déterminer si une expression est bien parenthésée, sans forcément utiliser une structure de données.

Son prototype sera : **`int checkParentheses(char *);`**

<u>Question 2)</u> proposez une fonction effectuant cette même vérification, en utilisant une pile – on utilisera, pour la pile, le type **`t_stack`**, en supposant qu'on y stocke bien des caractères

Etape 2: les crochets

Pour rendre plus lisibles certaines expressions, on utilise des crochets droits [et] en tant que 'super parenthèses'.
La vérification consiste maintenant à appliquer la même méthode pour les [ ] que pour les parenthèses, mais avec une exigence supplémentaire : les parenthèses et les crochets ne doivent pas être imbriqués…

Exemples :

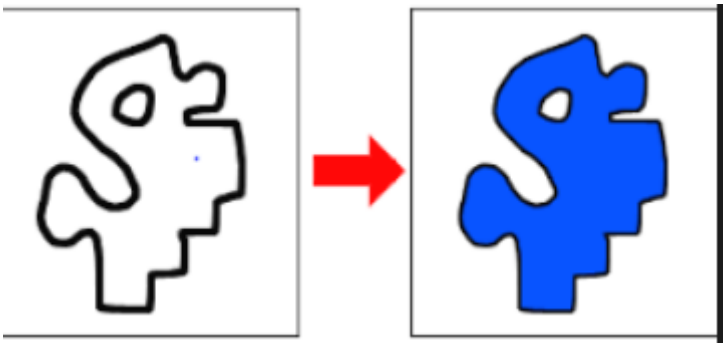
`[(x*y)/(3+a)] - (x²+x-4)`	est bien parenthésée
`[[([0])]]`	est bien parenthésée
`(5-z) * [(5+z²])`	ne l'est pas, la **`]`** est mal placée

Question 3) proposez une fonction qui vérifie la cohérence du parenthésage avec les [] en utilisant une seule pile

Exercice 6) utilisation d'une file pour remplir une forme avec une couleur (flood fill).

On souhaite effectuer l'opération de 'flood fill', pour remplir une zone d'une image avec une couleur unie : on remplace une couleur par une autre.

Illustration : remplissage de l'intérieur de la forme – on remplace tous les pixels blancs par des pixels bleus



On suppose que l'image est représentée par des pixels de type **`t_pixel`**, qui offre les fonctionnalités suivantes :

Le type **`t_color`** est un type énuméré

`t_color getColor(t_pixel);` : indique la couleur d'un pixel

`void setColor(t_pixel*, t_color)` : change la couleur d'un pixel

`t_pixel pixelUp(t_pixel)` : fournit le pixel 'au dessus' d'un pixel

de même pour : **`pixelDown(t_pixel)`**, **`pixelLeft(t_pixel)`**, **`pixelRight(t_pixel)`**

`int isInImage(t_pixel);` indique si un pixel est bien dans l'image (en testant ses coordonnées par exemple)

La méthode de flood fill est la suivante :

- On donne un pixel P de départ, et la couleur de remplacement.
- La couleur à remplacer est celle du pixel fourni.

L'idée est d'utiliser une file dans laquelle part du pixel de départ, puis on traite ses voisins, et ainsi de suite….

Voici le début de la fonction, à vous de la compléter : le type pour utiliser une file est **`t_queue`**, vous pouvez utiliser **`enqueue`**(), **`dequeue`**(), **`top`**(), **`isEmptyQueue`**()

```
void FloodFill(t_pixel P, t_color newcolor)
{
    t_queue pixqueue;
    t_pixel pixel, voisin;
    t_color oldcolor;

    pixqueue = emptyQueue();
    enqueue(&pixqueue, P);  /* la file est modifiée */
    oldcolor = getColor(P);
```